

# The schema behind the memory.

Relational data structure for ShiftForge — designed to keep every data point locked to its correct machine, crew, shift, and permit. No mixing up. This is the foundation Casey builds against.

## Core entities (7 tables)

| Table          | Purpose                                                          | Primary key |
|----------------|------------------------------------------------------------------|-------------|
| sites          | Client mine sites / commercial premises                          | site_id     |
| locations      | Location hierarchy — level, area, position within site           | location_id |
| assets         | Every piece of equipment · unique digital twin                   | asset_id    |
| shifts         | Every shift instance · day/afternoon/night with date + site      | shift_id    |
| reports        | Every shift handover report submitted                            | report_id   |
| report_assets  | Junction table · links reports to specific assets (many-to-many) | id          |
| users          | Personnel · fitters, supervisors, engineers, managers            | user_id     |
| permits        | Active permits-to-work · isolation records                       | permit_id   |
| parts          | Parts inventory + consumption log                                | part_id     |
| service_events | Every service performed on an asset (preventive + corrective)    | event_id    |

## Key relationships

- **sites** ← 1:many → **locations** (a site has many locations)
- **sites** ← 1:many → **assets** (a site has many assets)
- **locations** ← 1:many → **assets** (a location can hold many assets)
- **assets** ← 1:many → **service\_events** (an asset has many services over time)
- **shifts** ← 1:1 → **reports** (one report per shift)
- **reports** ← many:many → **assets** (a report can affect many assets; an asset appears in many reports)
- **assets** ← 1:many → **permits** (an asset may have many isolation permits over time)
- **users** ← 1:many → **reports** (a user files many reports)

## assets table · full schema

- ✓ asset\_id · UUID · primary key
- ✓ site\_id · foreign key → sites
- ✓ location\_id · foreign key → locations
- ✓ asset\_code · TEXT · human-readable (e.g. "C32-01")
- ✓ description · TEXT ("Cat C32 Conveyor Drive Motor")
- ✓ make · TEXT
- ✓ model · TEXT
- ✓ serial · TEXT · unique per asset
- ✓ discipline · ENUM (mechanical, electrical, hydraulic, pneumatic, structural, instrumentation)
- ✓ custodian\_crew · TEXT

- ✓ install\_date · DATE
- ✓ expected\_service\_life\_hours · INTEGER
- ✓ current\_service\_hours · INTEGER (rolling counter)
- ✓ warranty\_expires · DATE (nullable)
- ✓ health\_score · DECIMAL(3,2) (0.00 to 1.00 · calculated)
- ✓ status · ENUM (operational, under\_maintenance, offline, decommissioned)
- ✓ manufacturer\_url · TEXT (spec sheet / manual link)
- ✓ spare\_parts\_bom · JSONB (linked parts + qty required)
- ✓ metadata · JSONB (extensible per-manufacturer specs)
- ✓ created\_at · TIMESTAMP
- ✓ updated\_at · TIMESTAMP

## reports table · full schema

- ✓ report\_id · UUID · primary key
- ✓ shift\_id · foreign key → shifts
- ✓ site\_id · foreign key → sites
- ✓ author\_user\_id · foreign key → users (the fitter who submitted)
- ✓ report\_type · ENUM (shift\_report, incident, service\_log, inspection)
- ✓ priority · ENUM (low, medium, high, critical)
- ✓ work\_completed · TEXT
- ✓ issues\_identified · TEXT
- ✓ actions\_taken · TEXT
- ✓ parts\_used · TEXT (or JSONB link to parts consumption)
- ✓ delays · TEXT + JSONB (categorised: permits, isolations, access, parts, weather)
- ✓ current\_status · TEXT (auto-derived from asset statuses touched)
- ✓ planned\_work\_next\_shift · TEXT
- ✓ recommendations · TEXT
- ✓ ai\_generated\_narrative · TEXT (the polished report from AI)
- ✓ human\_edited · BOOLEAN (did fitter modify AI draft?)
- ✓ signed\_off\_at · TIMESTAMP
- ✓ distributed\_to · JSONB (array of user\_ids sent to)
- ✓ acknowledged\_by · JSONB (array of user\_ids who read + acknowledged)
- ✓ created\_at · TIMESTAMP

## report\_assets junction table · full schema

This is the critical table that keeps data un-mixed. Every report that touches an asset creates a row here. Query "give me all reports for asset X" or "give me all assets in report Y" — both directions supported.

- ✓ id · UUID · primary key
- ✓ report\_id · foreign key → reports
- ✓ asset\_id · foreign key → assets
- ✓ role · ENUM (primary\_focus, mentioned, indirectly\_affected)
- ✓ asset\_specific\_notes · TEXT (per-asset context within the report)
- ✓ fault\_flagged · BOOLEAN
- ✓ fault\_severity · ENUM (info, minor, major, critical) (nullable)
- ✓ resolved\_in\_this\_report · BOOLEAN
- ✓ created\_at · TIMESTAMP

## Row Level Security (RLS) — role-based access

Postgres RLS enforces access control at the database layer. Every query is filtered by user role + site membership. No app-layer bugs can leak data.

| Role         | What they can see                                         | What they can edit                          |
|--------------|-----------------------------------------------------------|---------------------------------------------|
| fitter       | Own site · own reports · assets in their discipline       | Own reports (until signed off)              |
| supervisor   | Own site · all reports · all assets                       | All reports (approve/reject) · asset status |
| engineer     | All sites they're assigned · all reports · full analytics | Recommendations · asset spec updates        |
| site_manager | Full site · full analytics · financial data               | Everything except system config             |
| admin        | Everything across all sites                               | Everything                                  |

## The "no mixing up" guarantee

Every data point in ShiftForge has:

- ✓ A specific asset foreign key (or joined via report\_assets)
- ✓ A specific site foreign key
- ✓ A specific author user\_id
- ✓ A specific timestamp
- ✓ A specific shift context
- ✓ Row-level security enforcing role-based access

Meaning: query "show me every hydraulic fault on asset C32-01 in the last 60 days" and you get exactly that. Not a fuzzy match, not a keyword search — a precise structured retrieval. That's what makes ShiftForge worth \$80-150/user/mo. That's what a spreadsheet can never do.